

Acing the MongoDB Viva

Projection, Modifiers & Update Operators

```
db.collection.find({}, { projection: { status: 1 } })
```

```
db.collection.updateOne({}, { $set: { status: "active" } })
```

```
$push: { array: value }
```

```
$inc: { counter: 1 }
```

▶ Basics

👜 Interview Ready

⚠️ Viva Ready

🔥 The Grilling

The Big Four Modifiers



Projection

Include or exclude specific fields using binary logic (1 = show, 0 = hide).

```
db.students.find({}, {name: 1, age: 0})
```



Sort

Order the returned results in ascending (1) or descending (-1) order.

```
db.students.find().sort({age: -1})
```



Skip

Bypass a specific number of documents from the start of the result set.

```
db.students.find().skip(2)
```



Limit

Restrict the absolute total number of documents returned.

```
db.students.find().limit(2)
```

Real-World Application

Scenario A: Frontend Pagination

How do you load page 3 of a list, showing 10 items per page?

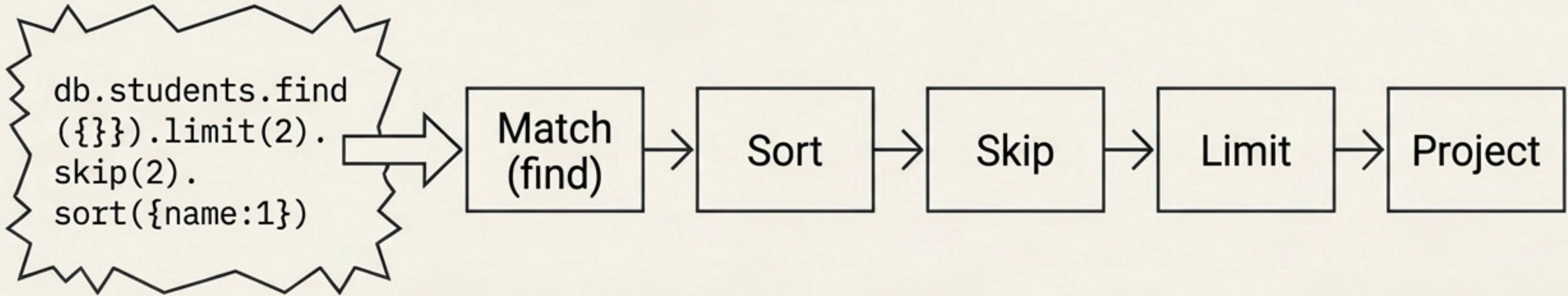
```
.skip(20).limit(10)
```

Scenario B: Data Privacy

How do you send user data without exposing passwords to the frontend?

```
db.students.find({}, {password: 0})
```

The Execution Pipeline



(Tangled Node.js Code)



Gotcha: Execution order is set in stone. Regardless of the order you chain the methods in your code, MongoDB always processes Sort first, then Skip, then Limit.

Memory Hacks & Shortcuts: Modifiers



Mnemonic 1: The Engine Rules

Order Doesn't Matter in Code.

Sort, Skip, and Limit happen naturally in their own hardcoded sequence inside the engine.

Write them however you want.



Mnemonic 2: The Stubborn Guest

The `_id` Exception

The `_id` field is the only field included by default. Limit does not hide it. To kick it out, you **MUST explicitly** request its removal using:

```
{_id: 0}
```



Mnemonic 3: Binary Switches

Projection Logic.

Think of projection as binary electrical switches.

- **1** = Show (or **Ascending** for sort).
- **0** = Hide.
- **-1** = Descending.

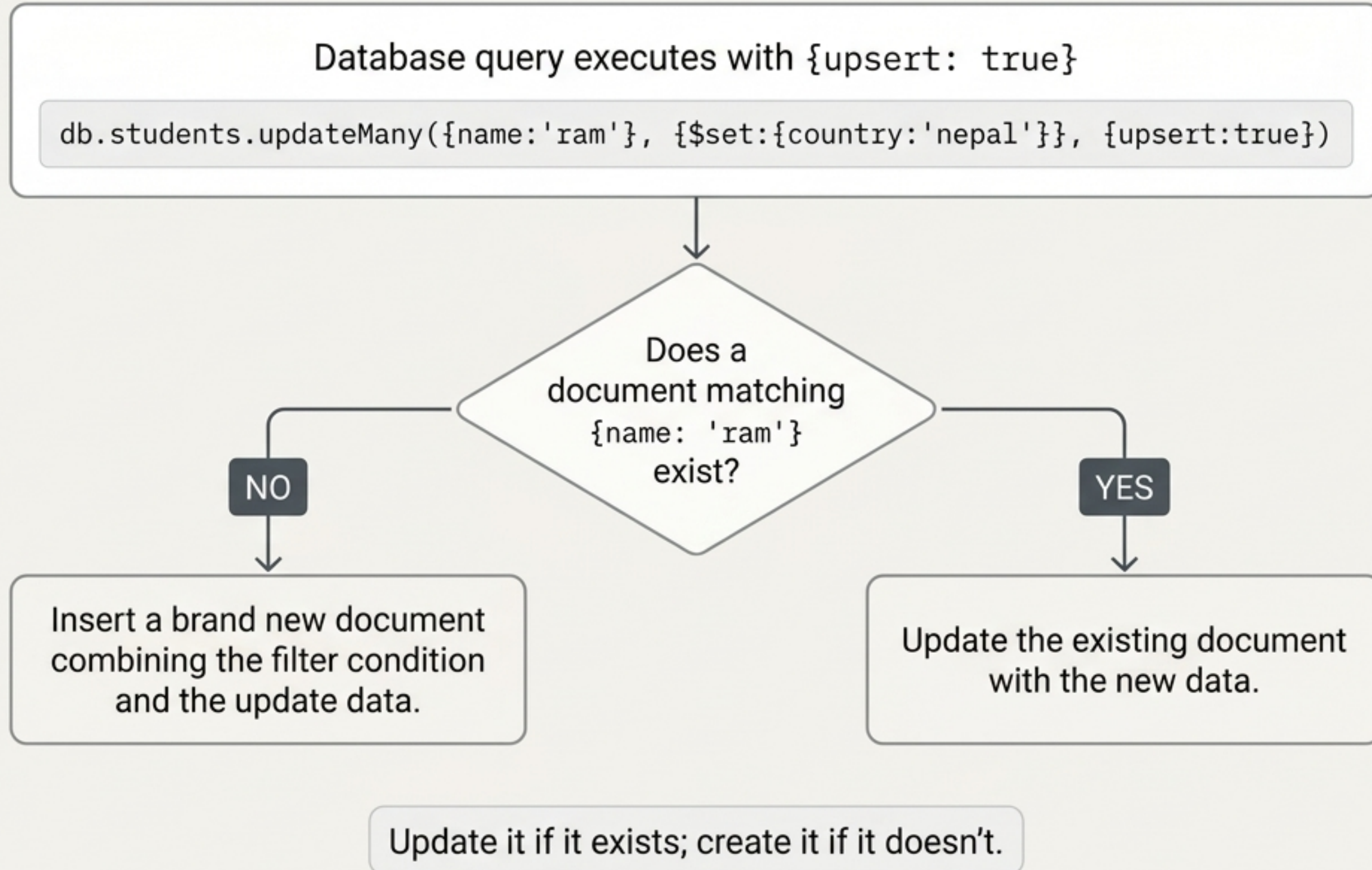
Basics: Field Modifiers

Operator	Plain-English Action	Exact Code Syntax
\$set	Overwrite or change an existing value.	<code>{\$set: {country: 'nepal'}}</code>
\$inc	Math: Increment or decrement a number.	<code>{\$inc: {age: 10}}</code>
\$unset	Completely delete the field/key.	<code>{\$unset: {city: ''}}</code>

⚠ Crucial Warning

Without update operators like \$set, passing a raw object to older MongoDB drivers replaces the ENTIRE document! Operators dictate targeted actions.

The Upsert Decision Tree

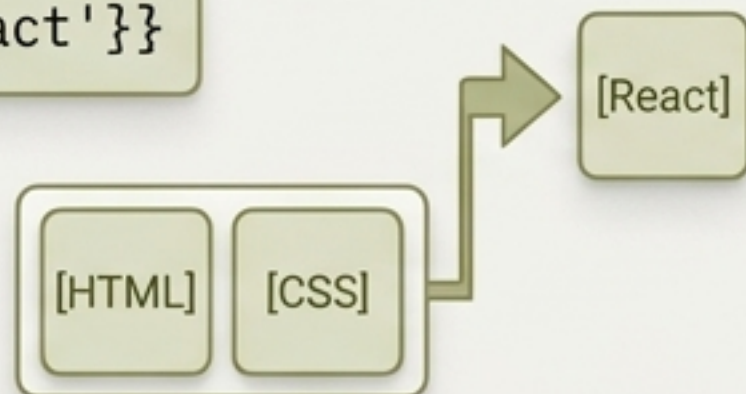


Basics: Array Operations

Adding Data

1. **\$push**: Appends a value to the end of the array.

```
{ $push: { skills: 'react' } }
```



2. **\$addToSet**: Appends uniquely—only if it doesn't already exist.

```
{ $addToSet: { skills: 'css' } }
```

Removing Data

1. **\$pull**: Removes specific matching values.

```
{ $pull: { skills: 'html' } }
```

2. **\$pop**: Removes the first (-1) or last (1) item.

```
{ $pop: { skills: 1 } }
```



3. **\$pullAll**: Removes multiple specified values simultaneously.

The Array Collision Matrix

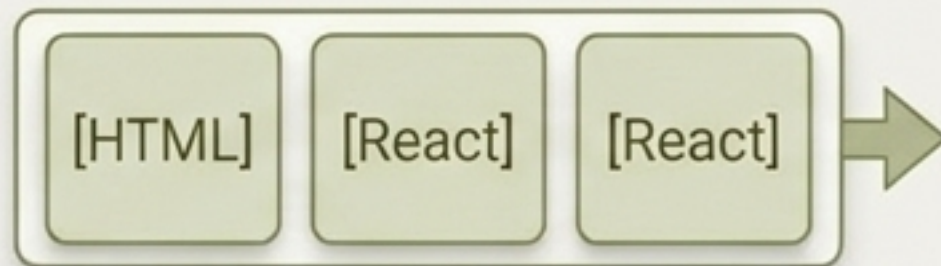
Setup

Starting Array: skills: ['HTML', 'React']

Action: We attempt to add 'React' to the array.

\$push

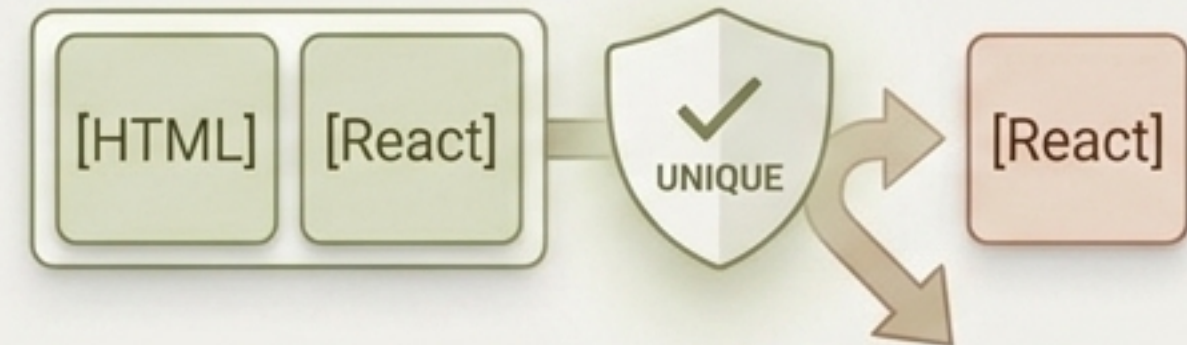
Action: Pushes 'React' directly into the array.



Result: Creates Duplicates.

\$addToSet

Action: Evaluates the array before adding 'React'.



Result: Strictly Unique.

🧠 **\$push** is pushy (takes everything). **\$addToSet** is a VIP bouncer (only new faces allowed).



The Hot Seat: Round 1 (Syntax Traps)

I want to return a document with the **student's name**, but I **do NOT want** the `_id` to show up. If I write:

```
db.students.find({}, {name: 1})
```

Will that work?

No.

The `_id` field is the only field included by default.

To suppress it, you must explicitly set it to 0. The correct projection object is:

```
{name: 1, _id: 0}
```



The Hot Seat: Round 2 (Logic Traps)

I have a collection and I run this exact query:

```
db.students.updateMany({name: 'ram'}, {country: 'nepal'})
```

What is the danger here? Why might this break our database?

You are missing the \$set operator.

Without a specific update operator, older MongoDB drivers will completely overwrite the existing document, deleting all other fields except 'country'.

You must ALWAYS use an operator to safely modify specific fields:

```
{ $set: { country: 'nepal' } }
```



The Final Boss (Execution Order Grilling)

Look at this chained Node.js code:

```
db.students.find().limit(2).skip(10)
```

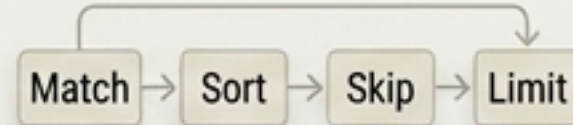
Does MongoDB limit the results to 2 and then try to skip 10?
What exactly is returned?

No.

MongoDB has a hardcoded execution engine order: Match -> Sort -> Skip -> Limit.
Regardless of how I chained them in the application code, MongoDB will ALWAYS skip the first 10 documents first, and then limit the output to the next 2 documents.

The Night-Before Cheat Sheet

Pipeline Rule



Match -> Sort -> Skip -> Limit.

Engine execution order is hardcoded.
Code chaining order does not matter.

Projection Rule

```
{ _id: 0 }
```

_id is the stubborn guest.

It is the ONLY field that requires an explicit 0 to be excluded from results.

Array Rule

\$push allows Duplicates.

\$addToSet is strictly Unique.

```
$push: { items: 'value' }
```

```
$addToSet: { items: 'value' }
```

Field Rule

Always use **\$set** to mutate data safely.

Operators dictate actions
(Set = overwrite, Inc = math,
Unset = delete key).

```
{ $set: { field: 'value' } }
```

Upsert Rule

```
{ upsert: true }
```

Update it if it exists; create it if it doesn't.

```
db.collection.updateOne({..},{..}, {upsert: true})
```

You're ready. Ace the Viva.